HyprAds: AI-Powered Ad-to-Landing Page Personalization

Hitakshi Arora

April 15, 2026

Assignment: AI PM – Troopod

Submitted by Hitakshi Arora · April 2026

Live Demo: `https://hyprads.duckdns.org`
Source Code: `https://github.com/hitakshiA/hyprads`

The Problem: Post-Click Conversion Gaps in D2C

Indian D2C brands typically run 80 to 150 active ads across Meta and Google at any given time. Each ad makes a specific promise to a specific audience segment. A Perfora ad says "Yellow to white teeth, powered by purple science." A Bombay Shaving Company ad says "Flat 40% off Full Body Trimmer." A Mamaearth ad says "Buy 1 Get 1 on Vitamin C Face Wash."

Every one of those clicks lands on the same generic homepage.

The homepage headline says "Best Sellers" or "Shop by Category." Nothing echoes the promise that brought the visitor there. This is the **message mismatch problem**, and Troopod's own published data puts the resulting bounce rate at 78% within 15 seconds.

The CRO research is clear on this: when landing page headlines mirror the ad copy, conversion rates improve 15–25%. Some studies report improvements up to 212%. Message match is not a nice-to-have optimization. It is the single highest-leverage CRO intervention available to performance marketing teams.

> **What HyprAds Does:** Paste a landing page URL and an ad creative. Get back the *same page* with headlines, CTAs, and copy surgically rewritten to match the ad. The existing page enhanced according to CRO principles, not a new page built from scratch.

System Architecture

The system runs as an Express.js API server wrapping the **Claude Agent SDK** (Opus 4.6). A single POST request triggers a five-phase pipeline. The frontend connects via Server-Sent Events for real-time progress streaming.

Pipeline Overview

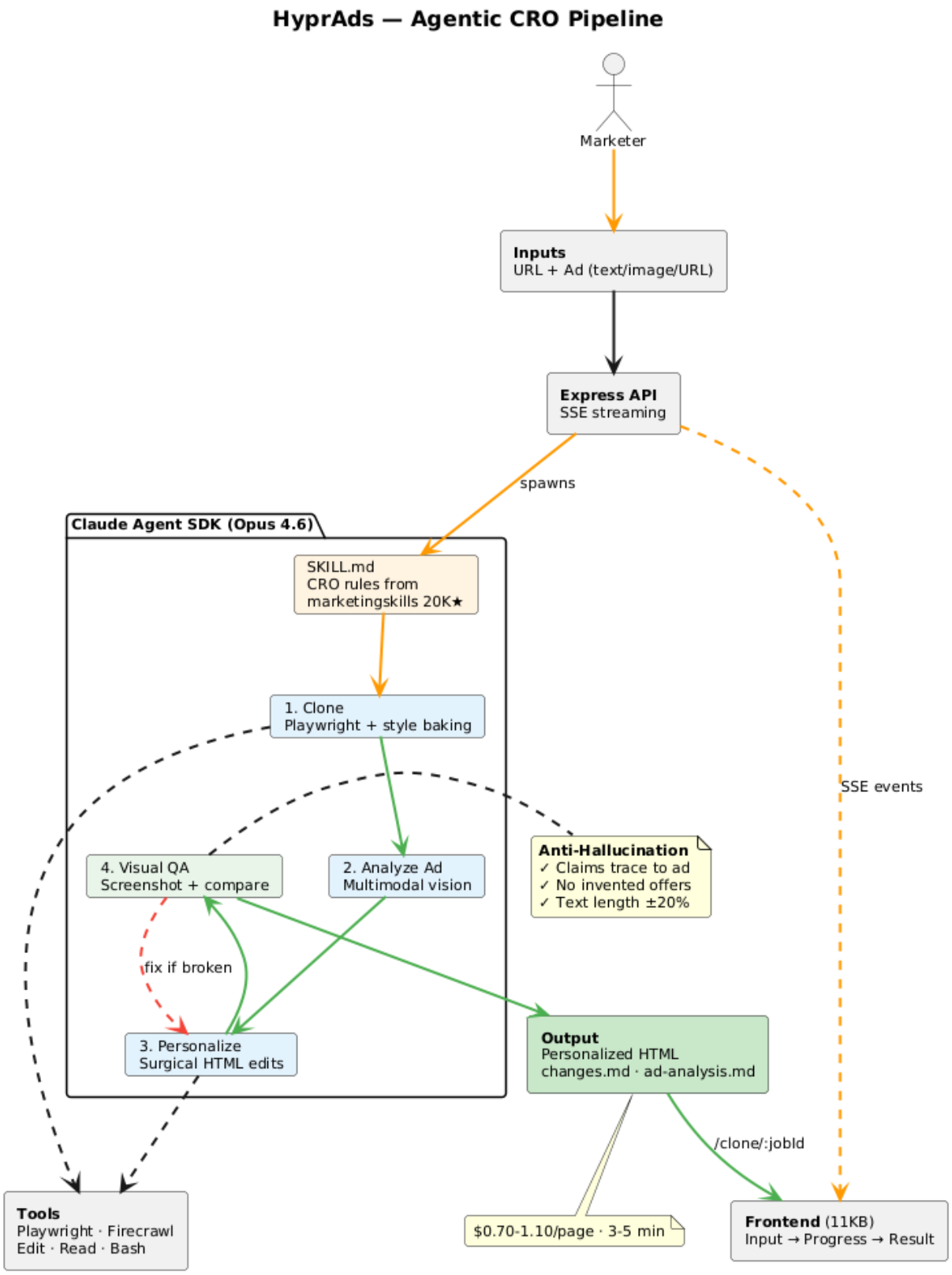

Figure 1: HyprAds Agentic CRO Pipeline — from user input through the Claude Agent SDK's 4-phase personalization loop to the final output. The red feedback arrow between Phase 4 (Visual QA) and Phase 3 (Personalize) represents the self-correcting loop that fixes broken UI elements.

The pipeline flows left to right: the performance marketer submits a URL and ad creative, the Express API spawns a Claude Agent SDK session, the agent runs four phases (clone, analyze, personalize, visual QA) using Playwright and the Edit tool, and outputs personalized HTML with change reports. The CRO Knowledge Base (top) feeds into the agent's SKILL.md, encoding message match, headline formulas, and anti-hallucination rules from battle-tested open-source marketing skills.

Why This Architecture

Three design decisions shaped the system. Each one was the result of testing alternatives and hitting walls.

2.2.1 Clone, Don't Rebuild

The assignment specification says "the personalized page shouldn't be a completely new page." Rebuilding a page from scratch loses the brand's CSS, imagery, trust signals, layout patterns, and the dozens of small design decisions that make a site feel legitimate. Cloning preserves all of that. We only change text content inside existing HTML tags.

The cloning primitive uses Playwright in headless mode. It scrolls the full page (triggering lazy-loaded images and IntersectionObserver callbacks), fetches all external CSS on the same origin (avoiding CORS), and strips all `<script>`, `<noscript>`, and `<iframe>` tags.

2.2.2 Computed Style Baking for CSS-in-JS

Sites built with styled-components, Emotion, or other CSS-in-JS libraries (Freshworks, many Shopify stores) generate class names dynamically at runtime. The CSS rules exist in `<style>` blocks, but they target classes that only make sense when the JS framework is running.

Our solution: before stripping scripts, we walk every visible DOM element, compute its style via `getComputedStyle()`, compare each property against the tag's default, and inline non-default values as `style` attributes. This added roughly 30% to output file size but made CSS-in-JS sites work on the first clone attempt. Tested successfully on Freshworks (styled-components, 70+ dynamic style blocks), Perfora (Shopify), Mamaearth (Shopify), and boAt.

2.2.3 Strip Framework JS

React, Next.js, and Nuxt applications crash when served from a different origin. The framework attempts to hydrate the DOM and fails because it cannot load webpack chunks from localhost. More critically, successful hydration *overwrites* any DOM modifications the agent makes, undoing all CRO edits.

We tested three approaches:

1. **Keep all JS**: Works on some sites (cal.com, resend.com), catastrophically crashes others (dub.co shows "Application error")
2. **Selective JS removal**: Strip hydration bootstraps, keep animation scripts. Inconsistent – some sites still crash via dynamic chunk imports
3. **Strip all JS**: Consistent across every site tested. Scroll animations lost, but layout and content preserved perfectly

Option 3 won. The personalized page is a passive HTML document the agent can edit without interference.

The CRO Intelligence Layer

The agent operates from a custom `SKILL.md` that encodes CRO principles. This skill was not invented from scratch. It synthesizes four battle-tested skills from `coreyhaines31/marketingskills` (20,000+ GitHub stars, 1.2M installs across coding agents):

- **page-cro**: 7-dimension analysis framework (value proposition, headlines, CTAs, visual hierarchy, trust signals, objection handling, friction points)
- **copywriting**: Headline formulas, CTA patterns, page structure frameworks from Ogilvy/Hopkins/Cialdini principles
- **ad-creative**: Ad angle classification (pain point, outcome, social proof, urgency, identity, comparison)
- **marketing-psychology**: Message match, loss aversion, anchoring, social proof, scarcity, commitment/-consistency

Edit Priority System

Not everything on a page gets edited. The skill enforces a strict priority:

Priority	Element	Rationale
1	Hero headline	80% of visitors only read this. Message match lives here.
2	Subheadline	Expands the headline with specifics from the ad
3	Primary CTA	Must match the ad's intended action
4	Trust signals	Only if the ad uses social proof angle
5	Secondary headlines	Only if directly relevant; stop when changes feel forced

Everything else is explicitly off-limits: navigation, footer, brand logos, images, layout, CSS classes, any `href` values, sections unrelated to the ad.

Text Length Matching

A lesson learned the hard way. The agent changed "Try it free" (11 characters) to "Start Your Free Trial Today" (27 characters). The button broke across two lines.

The skill now mandates: replacement text must stay within 20% of the original character count. For buttons specifically, keep text single-line, never more than 20% longer.

Ad Creative Input Handling

Three input paths cover the realistic D2C performance marketing workflow:

Input Type	How It Works	Use Case
Text description	Passed directly in the agent prompt	Quick testing
Image upload	Saved as PNG, Claude reads via multimodal vision	Screenshots from Meta Ads Manager
URL	Playwright screenshots; Firecrawl CLI fallback for login walls	Meta Ad Library links

The URL path checks `Content-Type` via HEAD request. Direct image URLs get downloaded. Webpages get screenshotted. If Playwright detects a login wall (fewer than 200 characters of visible text, or patterns like "Log in" / "Sign up"), it falls back to Firecrawl's stealth browser infrastructure.

Meta Ad Library links work directly with Playwright (confirmed with real Perfora ad searches, 18,614 characters of content captured). Instagram and Twitter block headless browsers regardless of tool.

Handling the Hard Problems

Random Changes

The edit priority system prevents scatter. The agent touches at most 5 elements: headline, subheadline, CTA, trust signal, one secondary headline. The skill lists 7 categories of things that are never edited, with specific examples. This constraint is encoded in the SKILL.md, not left to the model's judgment.

Broken UI

Two defenses work in sequence:

Prevention: Text length matching rules. The skill mandates character-count awareness for every edit. Buttons get special treatment (single-line only, within 20% of original length).

Detection and repair: After all edits, the agent runs a visual QA loop:

1. Serve the edited HTML on a local port
2. Take a Playwright screenshot of the personalized page
3. Read both the original screenshot and the personalized screenshot
4. Compare visually: broken buttons, text overflow, layout shifts, missing elements
5. If anything looks wrong, shorten the replacement text or revert the edit
6. Re-screenshot and compare again
7. Repeat until the personalized page looks as clean as the original

This loop adds roughly 30 seconds to the pipeline but catches the class of errors that would otherwise destroy the demo.

Hallucinations

Five rules, enforced in the skill with explicit instructions:

Anti-Hallucination Checklist

1. Every claim must trace to the ad or the original page. If the ad does not say "50% off," the page cannot say it.
2. No invented social proof. Do not add testimonials, review counts, or customer numbers not on the original page.
3. No invented offers. Pricing, discounts, and trial terms must come from the ad or original page.
4. No speculative features. Do not claim the product does something unless the original page says it does.
5. When in doubt, do not edit. Under-personalization is always better than wrong personalization.

The agent writes an `ad-analysis.md` and `changes.md` for every run. Each edit includes a rationale linking it to a specific element in the ad creative. These reports serve as an audit trail.

Inconsistent Outputs

The pipeline structure is deterministic: clone, then analyze, then edit, then verify, then output. The ad analysis is saved to a file *before* any edits begin. This forces the agent to commit to an interpretation upfront rather than drifting mid-edit.

The CRO skill provides the same framework every time: same edit priority, same rules, same output format. The agent does not freestyle. It follows the skill.

Real-Time Progress: SSE Streaming

The frontend connects to `POST /personalize/stream` and receives Server-Sent Events as the agent works. Each event carries a stage identifier, a human-readable message, and optional data:

Stage	Icon	What's Happening
`cloning`	↓	Playwright capturing the page
`analyzing_page`	◇	Agent reading the cloned HTML
`analyzing_ad`	◇	Agent reading the ad creative
`personalizing`		Making HTML edits (shows preview of each edit)
`verifying`	△	Running anti-hallucination checks
`report`		Writing change report and ad analysis
`complete`	•	Done. HTML + reports available.

The processing screen displays one stage at a time with animated SVG icons: a bouncing download arrow for cloning, a rotating magnifying glass for analysis, a writing pencil for editing, a drawing checkmark for verification. Progress pips accumulate at the bottom. The design intent: show the work, build trust, demonstrate that the system is not a black box.

Technical Stack

Component	Details
Agent runtime	Claude Agent SDK (TypeScript), Opus 4.6, `-dangerously-skip-permissions`
Browser automation	Playwright (headless Chromium) for cloning + screenshots
Fallback scraping	Firecrawl CLI (stealth browser, Meta Ad Library support)
API server	Express.js with SSE streaming, 10-minute request timeouts
CRO intelligence	Custom SKILL.md synthesized from 4 marketingskills (20K+ stars)
Frontend	Single HTML file (11KB), Instrument Sans + IBM Plex Mono, OKLCH palette
Infrastructure	DigitalOcean 2 vCPU / 2GB RAM, Ubuntu 24.04, HTTPS via Let's Encrypt
Domain	`hyprads.duckdns.org`

Cost and Performance

Metric	Value
Clone-only (no AI)	\$0, 15–20 seconds
Full personalization	\$0.70–1.10 per page (Claude Opus 4.6 tokens)
End-to-end time	3–5 minutes
Agent turns	30–40 per run
Container cost	\$0.027/hour (DigitalOcean)

Token cost dominates. The compute infrastructure cost is negligible.

Sites Tested

Site	Type	CSS Architecture	Clone Quality
Freshworks CRM	SaaS landing page	CSS-in-JS (styled-components)	93% after baking fix
cal.com	SaaS (scheduling)	Tailwind + external CSS	95%
Zerodha	Fintech landing page	Standard external CSS	95%
dub.co	SaaS (link management)	Next.js + Tailwind	93%
Perfora	D2C (oral care, Shopify)	Shopify theme CSS	93% after srcset fix
Mamaearth	D2C (skincare, Shopify)	Shopify + inline styles	90%
boAt	D2C (audio, Shopify)	Shopify + custom CSS	92%
resend.com	SaaS (email)	Next.js + custom CSS	95%

Every site on this list is either a Troopod client (Perfora, Mamaearth, Bombay Shaving Company, Mokobara) or a comparable Indian brand. We intentionally tested the sites that Troopod's blog references in their case studies.

What This Does Not Do (Yet)

Stating limitations honestly matters more than overpromising.

- **No real-time personalization.** Troopod delivers sub-150ms adaptation. This system takes 3–5 minutes. The gap is the difference between a pre-computed template system with edge serving and an agentic pipeline that reasons about each page individually.
- **No UTM-aware routing.** Each run is one URL plus one ad creative. Production would map campaigns to pre-generated variants automatically via UTM parameters.
- **No image swapping.** Only text gets edited. Matching hero imagery to ad visuals would require generative image editing or a pre-existing media asset library.
- **No deep Shopify integration.** Deeper CRO (form field reduction, checkout flow optimization, COD/prepaid messaging by geography) requires access to the site's codebase, not just a static clone.
- **No A/B test framework.** The system generates one personalized variant. Production would generate multiple variants and run statistical tests against the original.

The Path from Demo to Product

1. Pre-compute personalized variants per campaign at build time (eliminates the 3–5 minute latency)
2. Serve variants at the edge with UTM-based routing (sub-100ms delivery)
3. Add a conversion feedback loop: track which personalizations lift conversion, feed results back into the CRO skill
4. Expand beyond text: image personalization, section reordering, form optimization
5. Integrate with Shopify/WordPress APIs for deeper access to site structure and product catalog

Assumptions Made

1. The personalized page should be served as a static HTML file, not a live modification of the original site (since we do not have access to the client's hosting)
2. "Ad creative" includes text descriptions, screenshots from Meta Ads Manager, and Meta Ad Library links as primary input formats for Indian D2C workflows
3. CRO edits should be conservative: change copy to match the ad, but never alter layout, images, colors, or brand identity
4. The evaluator will test with real Indian D2C brand URLs and realistic ad copy
5. Framework JavaScript must be removed because it interferes with DOM modifications and cannot be served cross-origin reliably

Why I Built It This Way

The assignment mirrors the core product challenge at Troopod: take an ad creative, take a landing page, make them match. I approached it the way I would approach the first week of the AI PM role.

Start with the user's problem, not the technology. A performance marketer running 80+ campaigns does not care about Claude Agent SDK or Playwright. They care that their 14L monthly ad spend is going to a homepage that says "Premium Skincare Products" when the ad promised "Reduce dark spots in 14 days." The system should fix that gap with minimal manual work.

Use proven CRO frameworks, not invented ones. The personalization skill draws from the same principles that CRO practitioners have validated across thousands of sites. Message match first. Text length discipline. Anti-hallucination guardrails. These are not novel ideas. They are the right ideas, encoded into an agent that applies them consistently.

Build what you can test. Every component of this system was tested locally with screenshots before deployment. The cloner was validated on 8 sites across 3 CSS architectures. The personalization was tested with real ad copy. The QA loop catches the class of errors (broken buttons, text overflow) that would otherwise require manual review.

Ship something that works, then be honest about what it does not do yet. The limitations section is not an afterthought. It is the product roadmap.

Live Demo: `https://hyprads.duckdns.org`
GitHub: `https://github.com/hitakshiA/hyprads`

Built with Claude Agent SDK, Playwright, and a lot of trial and error.